

Confidential Telemetry with MA-ABE, IPFS, and Windowed Bulletproof Range Proofs

Johan B

2026-01-18

Abstract

I present a protocol for privacy-preserving metric reporting that combines Multi-Authority Attribute-Based Encryption (MA-ABE) with a *windowed* zero-knowledge design based on Bulletproof range proofs. A provider maintains compact public state and produces *one proof per window* (for example, every 1–5 seconds) attesting that all committed metric values in that window lie within declared 32-bit ranges. Each record is committed and encrypted with associated data that binds provider, window, sequence, timestamp, schema, and a policy digest. To make per-record encryption practical and context-bound, MA-ABE encrypts a per-window seed Z ; a key-encryption key derived from Z and the *attestation hash* of the accepted header wraps the AEAD key K , so ciphertexts and keys are useless off-context and only authorized readers can decrypt. The system targets near-real-time ingestion and short proof windows; the proof cadence is per window, and deployments may buffer for transport within a window while keeping deterministic ordering by the canonical increasing-sequence rule.

Validators on a BFT chain verify the range proof for a window and the byte-exact header. All telemetry payloads are stored off-chain in IPFS as a single content-addressed raw block, referenced by a CID that is committed in the on-chain header. Validators fetch the committed IPFS object, verify its digest and structure, recompute per-record digests and the Merkle root over record commitments, and verify the Bulletproof proof over the commitments contained in that object. Only if verification succeeds is the window accepted and the provider allowed to publish wrapped key material bound to the accepted header. The chain pins a canonical proof suite identifier; providers choose policy parameters but not the proof rules. BFT quorum agreement anchors the accepted header and its attestation hash as the canonical truth for ordering and finality.

We centralize canonical encodings, domain separation, Merkle orientation and defaults, nonce discipline, transcript binding, and key-binding rules; provide algorithms for producing window payloads in IPFS and for two-phase window acceptance (commit then finalize); and specify deterministic validator logic for safety and ordering under BFT. The result complements existing collectors: transport remains familiar, while confidentiality is preserved end-to-end and correctness is elevated from trust in a collector to a third-party-verifiable proof per window.

All implementation-binding procedures are consolidated in the appendices (see Appendix, Algorithms A.1–A.7).

1 Introduction

Network monitoring is essential to operating production systems. Operators deploy tools such as Zabbix, Nagios, LibreNMS, and Prometheus to collect high-cardinality telemetry, trigger alerts, and investigate incidents. Agents and exporters report CPU load, latency, error rates, queue depths, and countless application metrics into a central collector that supports dashboards and SLOs. The model is effective and widely adopted, yet it concentrates trust and visibility in the

collector and its administrators: anyone with sufficient backend access can see everything, and endpoints can still submit falsified values that the collector dutifully records because correctness is not cryptographically enforced. In multi-domain environments—such as between suppliers and regulators, or across isolated network segments—sharing telemetry across domains requires tunnels, broker services, or manual procedures that either increase risk or erode fidelity. Mainstream systems typically secure telemetry with transport-level encryption (and sometimes encryption at rest), but they do not bind values to their context or provide verifiable, window-level guarantees about basic validity constraints. At the same time, modern protocols reduce passive visibility in the network, making it less feasible to compensate with on-path inspection.

This paper addresses three practical shortcomings of current monitoring: confidentiality of raw values, verifiable correctness of basic validity constraints, and a robust cross-domain transport that avoids trusted intermediaries. We keep the familiar workflow—agents continue to emit numeric metrics—but replace unverified trust with cryptographic checks and selective decryption. Concretely, each metric vector is committed with a digest that binds the provider, window, sequence, timestamp, schema, and a canonical policy digest, and is then encrypted for authorized readers. A Merkle root over the record commitments is published in an on-chain header. A single Bulletproof range proof per window attests that every committed metric value satisfies the declared 32-bit range constraints for that window. On chain, validators verify one proof and the header commitment per window under BFT consensus. Acceptance acts as a gate: only after a window is accepted may the provider publish the wrapped key material that enables authorized readers to decrypt the corresponding off-chain payload.

My contribution is a specification that complements existing collection pipelines rather than replacing them. On the data plane, producers emit the same metrics they emit today; on the control plane, headers, proofs, and bindings make those metrics verifiably range-valid for their declared policy before they qualify for storage or attention. Authorized readers can later decrypt and recompute commitments to check equality with the accepted root; anyone can perform a non-decrypting availability and integrity check for payload structure and Merkle-root equality; and challengers can present a violating record, its commitment openings, and a Merkle path to trigger slashing in deployments that choose to support disputes. The result is a monitoring workflow that preserves operational ergonomics while adding cryptographic safeguards appropriate to encrypted, multi-party networks.

2 Background and Threat Model

2.1 Network Monitoring Systems

Network monitoring systems collect measurements about machines and applications so operators can keep systems healthy. Typical measurements include CPU and memory usage, request latency, error rates, queue lengths, and network throughput. Over time, these systems have expanded from “just metrics” into a broader picture that also includes logs, traces, and discrete events. The goal is practical: detect problems early, debug incidents, and plan capacity before outages happen.

In most real deployments, data is collected in steps. Small agents run close to workloads and send measurements to a collection service. That service may batch data, apply basic cleanup rules (for example, trimming obviously sensitive fields), and handle backpressure when load spikes. Doing some work early keeps later storage systems stable and reduces cost. It also helps when monitoring crosses boundaries (data center to cloud, one team to another, supplier to regulator), because it is easier to enforce a small set of stable rules at the edge than to rely on a single “trusted central brain.”

2.2 Centralized Models and Their Limitations

Many monitoring setups use a central collector (or a small collector cluster) that receives measurements from many machines and stores them in one place. This is convenient because operators get one dashboard, one query system, and one alerting pipeline.

The problem is that central collectors become a bottleneck as systems grow. More machines and services means more incoming data, and spikes during incidents are common. Even if the average load is fine, short bursts can overload the collector, delay alerts, and cause dropped samples. A central collector is also a single point of failure. If it is misconfigured, goes down, or is attacked, operators lose visibility exactly when they need it most.

A second scaling problem comes from *too many unique time series*. In systems like Prometheus, each unique combination of labels creates a separate time series. If someone attaches values like request IDs, user IDs, or rapidly changing container hashes as labels, the system may end up trying to track millions of distinct series. That uses a lot of memory and can slow down ingestion and queries. Teams usually work around this by limiting which labels are allowed, grouping values into categories, or aggregating high-rate signals before they reach the main database.

When a single cluster no longer works, two common approaches appear. One is to funnel everything into a larger backend, which keeps things simple but increases the impact of failures at the ingest tier. Another is to run multiple backends and query across them, which spreads ingestion but can make queries slower because the overall query is as slow as the slowest shard. Neither approach solves the trust issue created by a central place that both sees and controls all data.

2.3 Security and Privacy Shortcomings

Traditional monitoring systems usually protect telemetry with encryption in transit (and sometimes encryption at rest). That blocks simple eavesdropping, but it does not stop a compromised endpoint from sending fake numbers. The collector typically has no cryptographic way to check whether values are within reasonable bounds unless it fully trusts the sender or decrypts everything and applies checks centrally.

Access control is often coarse. Administrators of the monitoring backend can usually see all collected data. Telemetry can reveal sensitive details, such as service structure, traffic patterns, internal names, or user behavior. If an attacker or insider gains privileged access, they may read, delete, or manipulate metrics and logs. In addition, modern network encryption reduces what can be safely inspected in the middle of the network, which makes it harder to rely on passive observation or middleboxes for policy enforcement.

A practical alternative is to move assurance to the producer side and make verification possible for third parties. Each record is bound to its context so it cannot be replayed or relabeled without detection. Values are encrypted end-to-end, and a compact proof per window lets validators confirm basic validity constraints (range checks) without learning plaintexts. Readers only obtain decryption material after a window has been accepted.

2.4 Adversary and Assumptions

We assume compromised or misconfigured endpoints that may emit falsified metrics; honest-but-curious infrastructure operators and observers; and network partitions or censorship attempts. Confidentiality targets prevent unauthorized observers from learning plaintext metrics; correctness targets ensure that accepted windows satisfy the declared 32-bit range predicate for all committed values. We trust the cryptographic primitives (MA-ABE, AEAD, BLAKE3, Pedersen commitments, and Bulletproofs) and the BFT consensus for ordering and finality.

A realistic adversary also exploits operational weaknesses. For example, an attacker may try to overload monitoring by creating too many distinct series, or may try to confuse ordering by drifting timestamps or sequence numbers. These behaviors are mitigated when sequence and window identifiers are enforced monotonically, when window acceptance is tied to a proof over exactly the committed payload, and when validators verify deterministic bindings. Within these constraints, the security goal is twofold: unauthorized parties and intermediaries should learn only the small set of public fields needed for routing and verification, never the metric values themselves; and every committed metric value in an accepted window should satisfy the published range constraints, with any attempt to replay, relabel, or attach keys off-context detected and rejected. The architecture separates transport and storage from trust by making acceptance contingent on third-party-verifiable artifacts rather than on the good behavior of a central repository.

3 Cryptographic Preliminaries

This section fixes the building blocks and conventions that make independently written implementations derive the same bytes and public inputs. The prose below is explanatory. The authoritative reference for byte-accurate rules is Algorithm A.1.

Provider identity. The provider identifier `pid` is a 32-byte value registered on chain. A deployment SHOULD define `pid` as a stable identifier derived from the provider’s registration public key (for example, a 32-byte hash of the public key and chain id under a fixed domain separator). This paper treats `pid` as an opaque 32-byte identifier.

Two-domain cryptography. The protocol uses two cryptographic domains: MA-ABE uses a pairing-friendly curve for attribute-based encryption of the per-window seed Z (off-chain); commitments and Bulletproof range proofs use a prime-order group $\mathbb{G}$ (defined below). Cross-domain bindings (headers, attestation hash, transcript binding, KEK derivation) are done via byte hashes (BLAKE3 and HKDF) and are therefore independent of the ABE curve.

MA-ABE as a black box (interface). We use MA-ABE as a black-box primitive only to distribute a per-window seed Z to authorized readers under an access policy $\mathcal{P}$. We assume standard semantic security (IND-CPA) for ciphertexts and collusion resistance across authorities, as in prior MA-ABE work. The protocol encrypts only Z ; all telemetry records remain encrypted under a symmetric per-window AEAD key K , and key release is gated on window acceptance. The seed Z MUST be sampled uniformly at random as 32 bytes and MUST be unique per (pid, wid) . Deployments treat MA-ABE public parameters (authority keys and attribute namespaces) as configuration, referenced by stable identifiers; the chain does not validate MA-ABE ciphertexts.

Commitment group and range widths. We fix $\mathbb{G}$ to a prime-order group suitable for Bulletproofs. The default instantiation uses the Ristretto255 group derived from Curve25519. Each metric value is represented as an unsigned 32-bit integer. Policies bind the allowed range of each metric component using 32-bit bounds and are expressed as an application-level vector θ that is hashed into `PolicyHash` and used to bind headers, AEAD associated data, and Bulletproof transcripts. Policies MUST satisfy $0 \leq \ell_j \leq u_j \leq 2^{32} - 1$ for every metric index j .

Commitments. Each metric component M_j is committed with a Pedersen commitment $C_j = M_jG + r_jH$, where $G, H \in \mathbb{G}$ are fixed generators and r_j is a per-value blinding scalar. Commitments are stored off-chain and are included in the IPFS payload that validators and readers fetch by CID.

Window-level range proofs. For each window, the provider produces a single Bulletproof range proof π_{win} that covers all committed values in that window. To support policy-specific bounds $[\ell_j, u_j]$ per metric component, the proof is constructed over *derived commitments* that are computable from the published commitments and the public bounds: for each commitment C to M , validators form $C^{\text{lo}} := C - \ell \cdot G$ (a commitment to $M - \ell$ with the same blinding) and $C^{\text{hi}} := u \cdot G - C$ (a commitment to $u - M$ with negated blinding). Proving that both derived values lie in the 32-bit range implies $\ell \leq M \leq u$. The proof is bound to the committed header fields by seeding the Fiat–Shamir transcript with a deterministic header pre-hash and the policy digest. Validators verify π_{win} against the derived commitments computed from the payload commitments and policy bounds, and they accept a window only if the proof verifies and the Merkle root over recomputed record digests matches the root committed on-chain.

Hashing, Merkle trees, and payload integrity. We use BLAKE3-256 for all byte hashing in the protocol: payload digests, header hashes, and Merkle node hashes. The Merkle tree commits record digests rather than raw ciphertext bytes. A record digest binds the record’s public context and the list of commitments in that record. The complete records payload is stored off-chain in IPFS as a single raw block. The on-chain header commits to the payload by including both the payload CID and a BLAKE3 digest and size of the canonical payload bytes. The IPFS CID multihash (sha2-256) is used only for content addressing; the protocol’s canonical payload digest H_{batch} and all other byte hashing use BLAKE3-256.

Canonical helpers and encodings. We centralize byte layouts, domain separation strings, Merkle orientation and defaults, canonical payload layout, transcript binding, and header hashing in Algorithm A.1. This avoids divergence across implementations and makes validator behavior deterministic.

4 Formal Model

The provider maintains compact public state for each window and a private buffer that captures every checked update. Public state consists of a Merkle root over per-record digests, window-local bounds for timestamps, sequence bounds, the window length, and identifiers for provider, window, schema, and policy digest. The proof statement is a windowed range-validity claim: for every committed metric value referenced by the off-chain payload committed in the header, the value lies within the declared 32-bit bounds. The off-chain payload contains the commitments and ciphertexts; it is content-addressed and integrity-checked by hash and size. Canonical encodings, domain separation, Merkle orientation and defaults, and header packing are fixed in Algorithm A.1; record processing and proof generation are specified in Algorithm A.2 and Algorithm A.3.

Scope. The protocol verifies range constraints only. Relations between metrics, anomaly scoring, and higher-level predicates are computed by authorized clients after decryption, or by providers as additional derived metrics that can themselves be range-checked as ordinary 32-bit values. This keeps the verifiable statement small and uniform while preserving the ability to build richer analytics on decrypted data.

Purpose. At window end, a single proof certifies that every committed metric value in that window is range-valid under the declared policy, without revealing plaintexts. Commitments bind values to context through record digests and the Merkle root. Authorized readers later decrypt records and recompute commitments to check equality with the accepted root.

Mechanism. For each incoming metrics vector with timestamp and sequence, the provider samples commitment blindings, computes per-metric commitments, computes a record digest binding context and commitments, and updates a rolling Merkle root. Outside the proof system, the provider encrypts the plaintext record under a per-window AEAD key using canonical associated data that binds provider, window, sequence, timestamp, schema id, policy digest, and proof suite id. The tuple of record digest, commitments, ciphertext, sequence, and timestamp is appended to the off-chain payload buffer. At window end, the provider constructs a canonical payload byte string, stores it in IPFS as a single raw block, and commits its CID, size, and digest in the on-chain header. The provider then generates a Bulletproof range proof for all committed values referenced by that payload, binding the proof transcript to the header pre-hash and policy digest.

Usage. Agents emit vectors at a deployment-chosen cadence; the provider commits and encrypts each record and accumulates payload bytes locally. When the window ends, the provider uploads the payload to IPFS, commits the header, and finalizes with the range proof reference. Validators fetch the payload from IPFS, verify its digest and structure, recompute record digests and the Merkle root, and verify the range proof over the commitments. Authorized readers later decrypt records, recompute commitments, and aggregate digests to confirm equality with the accepted header.

5 Protocol

The on-chain protocol enforces a strict two-phase lifecycle—*commit* then *finalize*—followed by a post-acceptance *publish-keys*. Commit exposes a byte-exact header (**SERHDR**) that binds the window to an IPFS CID and payload digest but reveals no decryption material. Finalize verifies the window proof by fetching payload and proof from IPFS and checking deterministic bindings. Publish-keys is permitted only after acceptance and binds key material to the accepted header. Deterministic validator logic enforces ordering, parameter binding, and byte equality throughout. Canonical message layouts, hashes, transcript binding, and the IPFS payload format are defined in Algorithm A.1; the lifecycle is captured in Algorithms A.4, A.5, and A.8.

Flow. At window end the provider constructs the canonical records payload bytes B_{rec} , stores it in IPFS as a single raw block to obtain CID_{rec} , and serializes **SERHDR** with provider id, window id, sequence bounds, window length, timestamp bounds, Merkle root, schema id, policy digest, proof suite id, the payload CID bytes, and the payload digest and size. Validators check monotonicity of window identifiers and sequences and record the pending header. When the range proof is ready, the provider stores the proof object in IPFS to obtain CID_{π} and broadcasts **FinalizeWindow** carrying CID_{π} . Validators fetch the committed payload by CID_{rec} , verify its digest and structure, recompute per-record digests and the Merkle root, fetch the proof by CID_{π} , and verify the Bulletproof range proof against the derived commitments computed from the commitments in the payload and policy bounds, with a transcript bound to the header pre-hash and policy digest. On success they mark the window *Accepted* at a finalized height and compute **AttestHash** over the extended header bytes and acceptance context. Only after acceptance may the provider publish wrapped key material by calling **PublishKeys** with the ABE-wrapped seed and a wrapped per-window AEAD key.

Purpose. The protocol gates decryption material on a third-party-verifiable statement that the entire window satisfies the declared range policy, while preserving confidentiality of plaintext metrics. Headers are immutable, byte-exact commitments to the public state and to an off-chain IPFS payload; proofs are deterministic checks of commitment validity; key publication is context-bound via `AttestHash` so it is useless off-context.

Mechanism. Commit enforces strict monotone sequencing and binds the window to a specific off-chain payload via CID, digest, and size. Finalize fetches and checks the payload deterministically, recomputes per-record digests and the Merkle root, and verifies the range proof against the derived commitments computed from the commitments found in that payload under a transcript bound to the committed header. Publish-keys derives a KEK via HKDF-SHA256 from the ABE-decrypted seed and `AttestHash` and wraps the per-window AEAD key with associated data that binds the policy digest and `AttestHash`. In the base protocol, validators and the chain never handle plaintext metrics.

6 Off-Chain Retrieval, Availability, and Decryption

Authorized parties must be able to validate an accepted window without learning plaintext metrics and, when authorized, selectively recover plaintexts in a way that preserves binding to the accepted header. The protocol provides two complementary paths. A non-decrypting availability and integrity check verifies that the off-chain IPFS payload is complete and matches the accepted Merkle root and payload digest using only public bytes. An authorized retrieval path uses MA-ABE keys to recover a per-window seed, derives a KEK that is cryptographically tied to the accepted header, unwraps the per-window AEAD key, and then decrypts records with context-bound AAD so every recovered plaintext recomputes to its committed commitments and record digest and aggregates back to the published root. All byte layouts, encodings, and domain separators are canonical and fixed by Algorithm A.1; the procedures are captured in Algorithms A.6 and A.7.

Purpose. The availability path gives any observer a deterministic test for payload integrity, record ordering, and Merkle-root equality without access to secrets or plaintexts. The authorized path ensures that only readers with MA-ABE credentials learn the window seed and thus the KEK and AEAD key, while every decrypted record is context-checked through AAD, recomputed commitments, and recomputed record digests. Together, these paths decouple storage and transport from trust: correctness is verified against the accepted header, and confidentiality is enforced by construction rather than by backend privilege.

Mechanism. After a window is marked *Accepted*, validators have computed `AttestHash`. Anyone can fetch the payload by CID and perform a non-decrypting check by verifying the payload digest and size, parsing tuples deterministically, assigning window-local indices by record order, recomputing record digests and the Merkle root, and requiring equality with the value recorded in the accepted header (Algorithm A.6). This path leaks no plaintexts and needs no keys.

An authorized reader parses the same header and payload, decrypts the ABE-wrapped seed with MA-ABE keys, derives the KEK tied to `AttestHash`, unwraps the per-window AEAD key, and then decrypts each record under that key with the canonical AAD from Algorithm A.1. The reader requires that the recovered sequence and timestamp exactly match the public values, recomputes the commitments from the recovered plaintext metrics and the stored commitment randomness, and enforces equality to the commitments found in the payload. The reader then recomputes the record digest and the Merkle root and requires equality to the header. Any deviation—AAD mismatch,

re-commitment mismatch, record-digest mismatch, duplicate or missing indices, trailing bytes, or root inequality—causes verification to fail and the payload to be rejected.

7 Integration with Blockchain

The chain serves as a precise checkpoint for each reporting window. Rather than shipping raw telemetry, producers submit a compact header whose byte layout, domain separation, Merkle rules, and IPFS payload commitment are defined once in the canonical helpers (Algorithm A.1). A single range proof then certifies that the committed metrics referenced by that payload satisfy the declared 32-bit range policy. Validators check these artifacts once per window, comparing public fields byte-for-byte against the previously committed header and running the verifier selected by the proof suite identifier. This keeps on-chain work predictable while ensuring that all raw telemetry remains off-chain in IPFS and only becomes decryptable after acceptance and key publication. The commit/finalize handshake and its equality checks are specified in Algorithm A.4; the validator-side state machine and acceptance at a finalized height are in Algorithm A.8; the full provider-to-chain-to-IPFS path is collected in Algorithm A.5.

The window lifecycle follows a simple sequence. At the end of a window, a producer constructs a canonical payload byte string and stores it in IPFS. The producer then commits only the header, which binds the window to the payload CID and to a payload digest and size; validators record this intent and perform ordering and sequencing checks, but they accept no decryption material at this stage. When the range proof is ready, the producer stores the proof object in IPFS and finalizes by presenting its CID; validators fetch the payload and proof, recompute per-record digests and the Merkle root, and verify the proof under a transcript bound to the header. On acceptance at a finalized height, the chain derives an attestation hash that binds the accepted header to that height. Only then may the producer publish the wrapped keys. Under BFT consensus, acceptance reflects quorum agreement, anchoring the committed bytes and their verification result as canonical truth for readers and auditors.

Interoperability and evolution are handled explicitly so implementations do not diverge. A proof-suite registry maps a stable identifier to a pinned verification procedure and group parameters; producers choose the identifier at registration and cannot silently swap verifiers later (Algorithm A.8). Policy parameters are serialized canonically to a digest and written to an append-only policy registry; when policies change, the new digest is appended and future windows refer to it, while historical windows remain auditable against the exact parameters they committed to (Algorithm A.8).

IPFS storage model

All telemetry bytes are stored off-chain as a single IPFS raw block. The provider constructs the canonical payload byte string B_{rec} defined by Algorithm A.1 and stores it in IPFS as a CIDv1 block with multicodec **raw** and multihash **sha2-256**, obtaining CID_{rec} . The on-chain header commits CID_{rec} along with the application-level payload size $L = |B_{\text{rec}}|$ and payload digest $H_{\text{batch}} = \text{BLAKE3-256}(B_{\text{rec}})$. Validators MUST parse CID_{rec} and reject unless it is CIDv1 with multicodec **raw** and multihash **sha2-256**. Validators fetch by CID_{rec} and interpret the fetched bytes as B_{rec} directly, requiring $|B_{\text{rec}}| = L$ and $\text{BLAKE3-256}(B_{\text{rec}}) = H_{\text{batch}}$ before parsing and verification. This raw-block option requires that $|B_{\text{rec}}|$ fits within the deployment’s maximum block size; otherwise window parameters must be reduced. Multi-block payload layouts are out of scope for this version. For availability, deployments MUST ensure that CID_{rec} and CID_{π} objects remain retrievable for a policy-defined retention period (for example, by pinning on a validator subset or a designated

pinning tier); nodes that do not pin MUST NOT assume long-term retrievability of unpinned content.

8 Security Analysis

Confidentiality holds because metrics remain encrypted under a per-window AEAD key that is MA-ABE-protected; AAD binds the provider, window, sequence, timestamp, schema id, policy digest, and proof suite id; and commitments bind the same context to the committed values via record digests and the Merkle root. As a result, ciphertexts cannot be swapped across windows, schemas, or policies without detection. Integrity and range-correctness follow from Bulletproof verification against the derived commitments computed from the commitments fetched from IPFS and public policy bounds, with transcripts bound to the committed header and policy digest. Consistency between what was proved and what authorized readers decrypt is ensured by recomputing commitments from plaintext metrics using stored commitment randomness and requiring equality with the commitments in the payload, and by recomputing record digests and the Merkle root to match the accepted header. Freshness and ordering are enforced by monotone sequence indices and window identifiers in the public header and by BFT finality. Key gating ensures no decryption material is accepted unless the window is accepted.

Metadata leakage. The protocol provides confidentiality of metric *values*, not traffic-analysis resistance. Observers learn public window metadata required for routing and verification, including (pid,wid), sequence and timestamp bounds, the number of records N_{win} , schema and proof-suite identifiers, the policy digest `PolicyHash`, the committed payload ($\text{CID}_{\text{rec}}, L, H_{\text{batch}}$), and the fact that each record carries k commitments. The commitments and ciphertexts remain value-hiding under the assumed security of Pedersen commitments and AEAD, and commitment openings are available only to authorized readers because the blinding scalars are included only inside the AEAD-encrypted plaintext.

Omission resistance. The protocol proves that all values in the committed payload satisfy range policy; it does not by itself prove that all generated records were included. Deployments that require omission detection can bind agent-side signed counters or source-anchored append-only logs into the record digest and include those fields in the encrypted plaintext and associated data. This preserves the protocol structure while enabling higher-level accountability checks.

9 Benchmark Method and Results

This section reports the performance of the prototype implementation of our windowed proof pipeline. In each benchmark run we create a window containing N_{win} records, each record containing k integer metrics. For every metric we compute a Pedersen commitment, then we generate exactly one aggregated Bulletproof range proof for the entire window, and finally we verify that proof against the derived commitments computed from the window commitments and the public bounds. We record three timings: *setup* (commitment and window input construction), *prove* (window proof generation), and *verify* (window proof verification). The *total* time is the sum of these three phases.

The benchmark sweeps the two primary parameters that control proof size and work: k , the number of metrics per record, and N_{win} , the number of records per window. The reported times measure the cryptographic core only, namely commitment construction plus proof generation and verification, and therefore exclude network effects, IPFS fetch and pinning behavior, blockchain

consensus and mempool delays, and MA-ABE encryption or decryption overhead. The current implementation is also single-threaded, so the results should be interpreted as a conservative baseline rather than an optimized limit.

Figure 1 summarizes the benchmark sweep. The vertical axis is log-scaled to keep both small and large runtimes readable in one figure; it does not change the underlying measurements, it only changes how they are drawn.

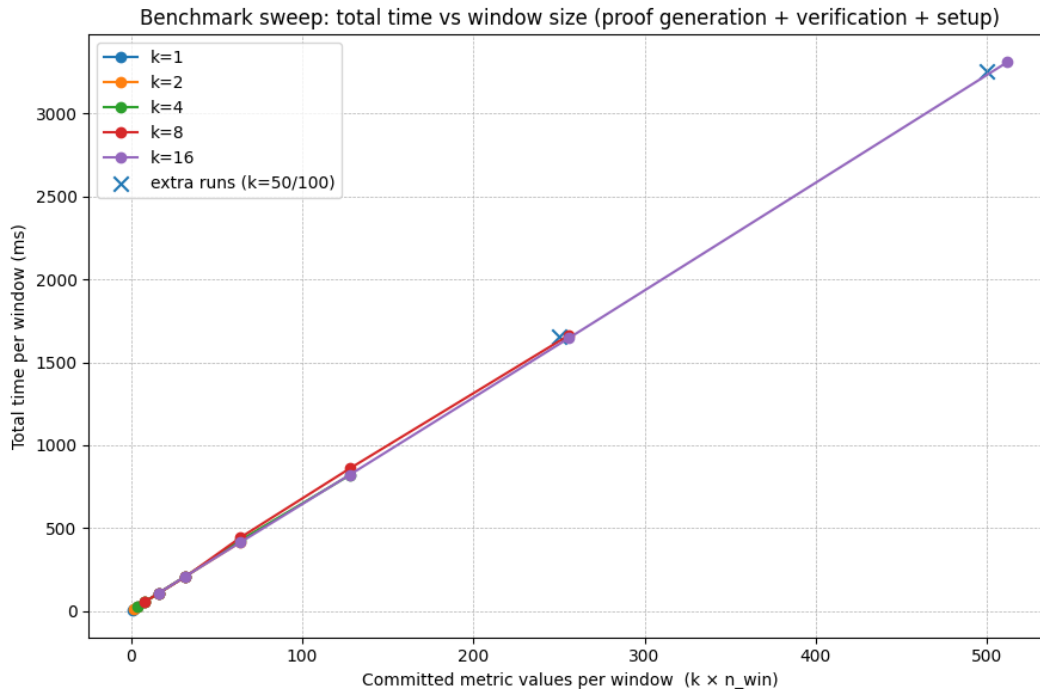


Figure 1: Benchmark sweep of the windowed proof pipeline. We vary k (metrics per record) and N_{win} (records per window) and measure setup time, proof generation time, and proof verification time. Each point corresponds to producing and verifying exactly one proof for a full window.

10 Conclusion

This design brings strong privacy and verifiable basic correctness into a domain that traditionally depends on trusted collectors. Agents keep their familiar workflow; providers introduce per-record commitments and encryption; validators check one proof per window by fetching off-chain bytes from IPFS; authorized readers decrypt only after acceptance; and auditors can recompute commitments and Merkle roots without privileged access. The combination of MA-ABE, IPFS, commitments, and Bulletproof range proofs achieves short proof windows and aligns incentives so that only verified data becomes decryptable and useful. The result is a trust-minimized, auditable system for decentralized metrics reporting that is rigorous enough for engineers and clear enough for decision-makers who are accountable for performance, privacy, and cost.

References

- [1] Melissa Chase. Multi-authority attribute based encryption. In *Theory of Cryptography Conference (TCC)*, 2007.

- [2] Melissa Chase and Sherman S. M. Chow. Improving privacy and security in multi-authority attribute-based encryption. In *ACM CCS*, 2009.
- [3] M. Bellare and C. Namprempre. The Authenticated Encryption with Associated Data (AEAD) Interface. *RFC 5116*, 2008.
- [4] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. *RFC 8439*, 2018.
- [5] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, Ittai Abraham. HotStuff: BFT Consensus in the Lens of Blockchain. In *PODC*, 2019.
- [6] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, Greg Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. In *IEEE Symposium on Security and Privacy*, 2018.
- [7] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO*, 1991.
- [8] Multiformats. CID: Content Identifier. <https://github.com/multiformats/cid>
- [9] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. 2020. <https://github.com/BLAKE3-team/BLAKE3>

Appendices

Algorithms (canonical, implementation-binding)

Implementation contract (brief). Encodings are big-endian with fixed byte widths. All byte hashing uses BLAKE3-256 and fixed ASCII domain separators. Merkle index bit order, sibling orientation, and default nodes are canonical. SERHDR byte layout is binding; validators compute header hashes and transcript seeds from SERHDR exactly as specified. AEAD uses 96-bit nonces; the canonical nonce is deterministically derived from the record order index per Algorithm A.2. The per-window AEAD key K must be unique per window. Only the per-window seed Z is ABE-encrypted; a KEK derived from Z , (pid, wid) , and AttestHash wraps K with associated data that binds PolicyHash and AttestHash . Payloads and proofs are stored off-chain in IPFS and are referenced on chain by CID. Validators fetch by CID and verify payload digests and sizes before parsing and verification. The records payload CID_{rec} MUST refer to a single CIDv1 raw block (multihash sha2-256) whose content bytes are exactly B_{rec} . Validators MUST parse CID_{rec} and reject unless it is CIDv1 with multicodec `raw` and multihash `sha2-256`. Parsers MUST reject trailing bytes: parsing MUST consume exactly L bytes.

Implementation Notes and Parameters. The deployment fixes the number of metrics per record k , the per-window maximum record count $N_{\text{win}, \text{max}}$, the maximum payload size per window, and an upper bound L_{max} on $|B_{\text{rec}}|$ that must not exceed the network’s maximum block size. Each metric is a 32-bit unsigned integer. Policies define per-metric 32-bit bounds that are serialized canonically and hashed into PolicyHash . Policies MUST satisfy $0 \leq \ell_j \leq u_j \leq 2^{32} - 1$ for all metric indices j . Commitments and Bulletproofs use a prime-order group $\mathbb{G}$, with the default instantiation using Ristretto255. The Merkle tree depth D is fixed as $D = \lceil \log_2 N_{\text{win}, \text{max}} \rceil$. The proof suite identifier `proof_suite` pins group parameters, commitment encoding length, and the Bulletproof verification procedure, including the exact transcript procedure (transcript construction, labels, and absorb order); it is part of the header and AAD. Proof objects stored in IPFS MUST use the proof suite’s pinned canonical byte encoding.

MA-ABE usage (binding at the protocol boundary). MA-ABE is used as a black box only to encrypt the per-window seed Z under an access policy $\mathcal{P}$. The seed Z MUST be sampled uniformly at random as 32 bytes and MUST be unique per (pid, wid) . Implementations MUST treat MA-ABE public parameters (authority keys and attribute namespaces) as deployment configuration referenced by stable identifiers; the chain does not validate MA-ABE ciphertexts.

Proof suite instantiation (minimal pinned suite). A deployment MUST publish at least one proof suite in PROFSUITEREGISTRY. The default suite (`proof_suite` = 1) uses $\mathbb{G} = \text{Ristretto255}$, Pedersen commitments over $\mathbb{G}$, commitment encoding $\text{ENCC}(\cdot)$ as 32-byte compressed Ristretto, and an aggregated Bulletproof range-proof verifier for 32-bit ranges. The suite pins the transcript construction and absorb order, and it MUST absorb `trSeed` as the first committed message under a suite-defined label.

Nonce and key requirements (binding). The AEAD suite is deployment-fixed and MUST use a 96-bit (12-byte) nonce. Providers MUST use a unique per-window AEAD key K ; K MUST NOT be reused across windows or providers. The canonical per-record nonce is $\text{I2OSP}(\text{winIndex}, 12)$

where $\text{winIndex} = i - 1$ is the record order index in the window. Therefore deployments MUST enforce $N_{\text{win,max}} < 2^{96}$ so $\text{winIndex} < 2^{96}$ always holds for every record.

Key wrapping (binding). The per-window key K is encoded as exactly 32 bytes. The key-wrapping ciphertext CT_K is produced using the same deployment-fixed AEAD algorithm (e.g., ChaCha20-Poly1305) under key $\text{KEK} = \text{DERIVEKEK}(Z, \text{pid}, \text{wid}, \text{AttestHash})$, with associated data $\text{enc}_{32}(\text{PolicyHash}) \parallel \text{AttestHash}$ and a fixed implicit nonce $\text{nonce}_K = 0x00 \times 12$. Using a fixed nonce for wrapping is safe only if KEK is unique per accepted $(\text{pid}, \text{wid}, \text{SERHDR}, \text{AttestHash})$; uniqueness is ensured by fresh Z per window and by binding (pid, wid) and AttestHash into DERIVEKEK .

Algorithm A.1 Canonical helpers (single source of truth)

- 1: **Define** byte hash BLAKE3-256 outputs 32 bytes.
 - 2: **Domain separators (ASCII):**
 $DS_{\text{leaf}} := \text{"maabe:leaf:v2"}; DS_{\text{node}} := \text{"maabe:node:v2"};$
 $DS_{\text{zero}} := \text{"maabe:merkle:zero:v2"}; DS_{\text{payload}} := \text{"maabe:payload:v2"};$
 $DS_{\text{attest}} := \text{"maabe:attest:v2"}; DS_{\text{hdrpre}} := \text{"maabe:hdrpre:v2"};$
 $DS_{\text{transcript}} := \text{"maabe:bp:transcript:v2"}; DS_{\text{policy}} := \text{"maabe:policy:v2"}.$
 - 3: **Define** $I2OSP(x, L)$: big-endian, fixed-length encoding to exactly L bytes; require $x < 256^L$.
 - 4: **Define** $enc_4(x) := I2OSP(x, 4)$, $enc_8(x) := I2OSP(x, 8)$, $enc_{32}(b)$: require $|b| = 32$ then return b .
 - 5: **Define** $POLICYHASH(\theta)$:
 let $b :=$ canonical byte serialization of θ (deployment-pinned ordering and widths); return $BLAKE3-256(DS_{\text{policy}} \parallel b)$.
 - 6: **Define** commitment encoding $ENCC(C)$:
 a fixed-length canonical encoding of a commitment $C \in \mathbb{G}$ pinned by the proof suite (e.g., 32-byte compressed Ristretto).
 - 7: **Define** enc_C_len : the fixed byte length of $ENCC(\cdot)$ pinned by the proof suite.
 - 8: **Define** Merkle orientation: write $winIndex = \sum_{\ell=0}^{D-1} b_{\ell} 2^{\ell}$ (little-endian bits, b_0 LSB). At level ℓ , if $b_{\ell} = 0$ use left child, else right child.
 - 9: **Define** $NODEHASH(L, R) := BLAKE3-256(DS_{\text{node}} \parallel L \parallel R)$ where L, R are 32-byte node values.
 - 10: **Define** default nodes: $Z_0 := BLAKE3-256(DS_{\text{zero}} \parallel 0x00)$ and $Z_{\ell+1} := NODEHASH(Z_{\ell}, Z_{\ell})$ for $\ell = 0, \dots, D-1$.
 - 11: **Define** $MERKLERECOMPUTE(winIndex, d, path[0..D-1])$:
 $x \leftarrow d$; for $\ell \leftarrow 0$ to $D-1$ do: if $b_{\ell} = 0$ then $x \leftarrow NODEHASH(x, path[\ell])$ else $x \leftarrow NODEHASH(path[\ell], x)$. Return x .
 - 12: **Define** $MERKLEAGGREGATE(\{(winIndex_i, d_i)\})$ (sparse input):
 require all $winIndex_i$ distinct and $0 \leq winIndex_i < 2^D$.
 Create an array $leaf[0..2^D-1]$ and set all entries to Z_0 . For each provided pair $(winIndex_i, d_i)$, set $leaf[winIndex_i] \leftarrow d_i$.
 For $\ell = 0$ to $D-1$: set $leaf[j] \leftarrow NODEHASH(leaf[2j], leaf[2j+1])$ for all j in $0..2^{D-\ell-1}-1$, overwriting in place.
 Return $leaf[0]$ as the root.
 - 13: **Define** record digest $RECDIGEST$:
 let $C_1, \dots, C_k$ be per-metric commitments encoded as fixed-length bytes via $ENCC(\cdot)$.
 $d := BLAKE3-256(DS_{\text{leaf}} \parallel enc_{32}(pid) \parallel enc_8(wid) \parallel enc_8(seq) \parallel enc_8(t) \parallel enc_4(schemalD) \parallel enc_{32}(PolicyHash) \parallel enc_4(proof_suite) \parallel ENCC(C_1) \parallel \dots \parallel ENCC(C_k))$.
 - 14: **Define** AAD bytes:
 $AAD(\cdot) := enc_{32}(pid) \parallel enc_8(wid) \parallel enc_8(seq) \parallel enc_8(t) \parallel enc_4(schemalD) \parallel enc_{32}(PolicyHash) \parallel enc_4(proof_suite)$.
 - 15: **Define** canonical payload B_{rec} :
 require strictly increasing $seq_1 < \dots < seq_{N_{\text{win}}}$, $N_{\text{win}} > 0$, and $|B_{\text{rec}}| \leq L_{\text{max}}$.
 require for all i : $|C_i| = k \cdot enc_C_len$ where k is deployment-fixed and enc_C_len is pinned by $proof_suite$.
 Set $B \leftarrow \text{"MAABE-PAYLOAD-V2"} \parallel (0x00 \times 16)$ as a 32-byte version tag, then append $enc_8(N_{\text{win}})$.
 for $i \leftarrow 1$ to N_{win} do append, in this order: $enc_{32}(d_i)$, $enc_8(seq_i)$, $enc_8(t_i)$, $enc_8(|C_i|) \parallel C_i$, $enc_8(|P_i|) \parallel P_i$, where C_i is the concatenation of $ENCC(C_{i,1}) \parallel \dots \parallel ENCC(C_{i,k})$ and P_i is the record ciphertext bytes.
 return B . Define payload size $L := |B|$ and payload digest $H_{\text{batch}} := BLAKE3-256(B)$.
 - 16: **Define** header bytes (extended):
 $SERHDR := enc_{32}(pid) \parallel enc_8(wid) \parallel enc_8(seq_{\min}) \parallel enc_8(seq_{\max}) \parallel enc_{32}(R) \parallel enc_8(t_{\min}) \parallel enc_8(t_{\max}) \parallel enc_8(N_{\text{win}}) \parallel enc_4(schemalD) \parallel enc_{32}(PolicyHash) \parallel enc_4(proof_suite) \parallel enc_8(L) \parallel enc_{32}(H_{\text{batch}}) \parallel enc_8(|CID_{\text{rec}}|) \parallel CID_{\text{rec}}$.
 - 17: **Define** header pre-hash:
 $hdrPre := BLAKE3-256(DS_{\text{hdrpre}} \parallel SERHDR)$.
 - 18: **Define** attestation hash:
 $AttestHash(SERHDR, chainID, h_{\text{accept}}) := BLAKE3-256(DS_{\text{attest}} \parallel enc_4(chainID) \parallel enc_8(h_{\text{accept}}) \parallel SERHDR)$.
 - 19: **Define** Bulletproof transcript seed:
 $trSeed := BLAKE3-256(DS_{\text{transcript}} \parallel hdrPre \parallel enc_{32}(PolicyHash))$.
 - 20: **Define** $proof_suite$ transcript rule:
 the proof suite pins the exact transcript procedure (construction, labels, and absorb order). The transcript MUST absorb $trSeed$ as its first committed message under a suite-defined label.
 - 21: **Define** KEK derivation:
 $salt = BLAKE3-256(\text{"maabe-streaming:win-key"})$. $info = \text{"win-key"} \parallel enc_{32}(pid) \parallel enc_8(wid) \parallel AttestHash$.
 $DERIVEKEK(Z, pid, wid, AttestHash) := HKDF-SHA256(Z, salt, info)$.
 - 22: **Define** key-wrapping nonce: $nonce_K := 0x00 \times 12$ and it is implicit.
-

Algorithm A.2 StreamingUpdate (commitments, payload buffer, and Merkle root)

Require: public constants for a window: (pid, wid, schemaID, PolicyHash, proof_suite)

Require: policy θ that hashes to PolicyHash under POLICYHASH and satisfies $0 \leq \ell_j \leq u_j \leq 2^{32} - 1$ for all j

Require: AEAD suite fixed; unique per-window key K ; nonce rule (canonical, deterministic) I2OSP(winIndex, 12)

- 1: **Initialization (per window):** set $\text{seq}_{\min} \leftarrow \infty$, $\text{seq}_{\max} \leftarrow 0$, $t_{\min} \leftarrow \infty$, $t_{\max} \leftarrow 0$, winIndex $\leftarrow 0$, and Merkle root $\text{root} \leftarrow Z_D$ where defaults Z_ℓ are from Algorithm A.1.
 - 2: **Inputs per record:** $\mathbf{M} \in \{0, 1, \dots, 2^{32} - 1\}^k$, $t \in [0, 2^{64})$, $\text{seq} \in \mathbb{N}$.
 - 3: **Commitments (out-of-chain; private witness for proof):** sample blindings $r_1, \dots, r_k$; compute per-metric commitments $C_1, \dots, C_k$ in $\mathbb{G}$.
 - 4: **Record digest (public):** compute $d \leftarrow \text{RECDIGEST}(\text{pid}, \text{wid}, \text{seq}, t, \text{schemaID}, \text{PolicyHash}, \text{proof_suite}, C_1, \dots, C_k)$.
 - 5: **Encrypt plaintext (out-of-proof):** nonce $\leftarrow \text{I2OSP}(\text{winIndex}, 12)$. Let plaintext be $(\mathbf{M}, r_1, \dots, r_k, \text{seq}, t)$. Compute $P \leftarrow \text{AEAD.Enc}_K(\text{plaintext}; \text{AAD}(\cdot), \text{nonce})$.
 - 6: **Update bounds and sequencing:** require $\text{seq} > \text{seq}_{\max}$ if winIndex > 0 . Update $\text{seq}_{\min} \leftarrow \min(\text{seq}_{\min}, \text{seq})$, $\text{seq}_{\max} \leftarrow \text{seq}$, $t_{\min} \leftarrow \min(t_{\min}, t)$, $t_{\max} \leftarrow \max(t_{\max}, t)$.
 - 7: **Merkle update:** treat winIndex as the leaf index. Compute new root by recomputing along a path for index winIndex using current defaults and cached frontier, producing root' . Set $\text{root} \leftarrow \text{root}'$.
 - 8: **Append to window buffer:** append tuple $(d, \text{seq}, t, C_1, \dots, C_k, P)$ to the in-memory buffer for this window.
 - 9: **Increment:** winIndex $\leftarrow \text{winIndex} + 1$.
-

Algorithm A.3 ProveWindowRanges (one Bulletproof per window)

Require: window buffer contains N_{win} records with tuples $(d_i, \text{seq}_i, t_i, C_{i,1..k}, P_i)$

Require: provider holds corresponding plaintext metrics and commitment blindings used to form commitments

Require: θ hashes to PolicyHash and specifies 32-bit bounds for each metric index, satisfying $0 \leq \ell_j \leq u_j \leq 2^{32} - 1$ for all j

- 1: **Canonical payload:** construct B_{rec} exactly as defined in Algorithm A.1. Let $L := |B_{\text{rec}}|$ and $H_{\text{batch}} := \text{BLAKE3-256}(B_{\text{rec}})$.
 - 2: **Store payload in IPFS:** store B_{rec} as a single CIDv1 raw block (multihash sha2-256) to obtain CID_{rec} .
 - 3: **Merkle root:** set $R \leftarrow \text{MERKLEAGGREGATE}(\{(i-1, d_i)\}_{i=1}^{N_{\text{win}}})$ with canonical defaults.
 - 4: **Header:** serialize SERHDR as in Algorithm A.1 using R , bounds, (L, H_{batch}) , and CID_{rec} . Compute hdrPre and trSeed as defined in Algorithm A.1.
 - 5: **Proof statement (policy bounds via derived commitments):**
For each metric commitment $C_{i,j}$ to $M_{i,j}$ with blinding $r_{i,j}$ and public bounds (ℓ_j, u_j) from θ , define
 $C_{i,j}^{\text{lo}} := C_{i,j} - \ell_j \cdot G$ (commitment to $M_{i,j} - \ell_j$ with blinding $r_{i,j}$),
 $C_{i,j}^{\text{hi}} := u_j \cdot G - C_{i,j}$ (commitment to $u_j - M_{i,j}$ with blinding $-r_{i,j}$).
Prove both derived values lie in the 32-bit range for all i, j .
 - 6: **Derived commitment list order (binding):**
Construct a single ordered list by iterating $i = 1..N_{\text{win}}$, then $j = 1..k$, and appending $C_{i,j}^{\text{lo}}$, then $C_{i,j}^{\text{hi}}$ for each (i, j) pair.
 - 7: **Transcript binding:** initialize the Bulletproof transcript exactly as specified by proof_suite, and absorb trSeed as the first committed message under the suite-defined label.
 - 8: **Generate Bulletproof:** compute $\pi_{\text{win}} \leftarrow \text{BP.Prove}(\{C_{i,j}^{\text{lo}}, C_{i,j}^{\text{hi}}\}, \{M_{i,j} - \ell_j, u_j - M_{i,j}\}, \{r_{i,j}, -r_{i,j}\}; 32\text{-bit}; \text{transcript})$ using the provider's witnesses.
 - 9: **Store proof in IPFS:** store the canonical proof bytes for π_{win} (pinned by the proof suite) as an IPFS object to obtain CID_π . Return $(\text{SERHDR}, \text{CID}_\pi)$.
-

Algorithm A.4 Commit and Finalize (off-chain verification by CID)

- 1: **Commit:** publish header bytes SERHDR for (pid, wid).

Require: $N_{\text{win}} > 0$. No key material prior to acceptance.

- 2: **Finalize:** publish CID_π for (pid, wid).
 - 3: **Validator fetch and checks:** parse CID_{rec} from SERHDR and require it is CIDv1 with multicodec raw and multihash sha2-256. Fetch B_{rec} by CID_{rec} from SERHDR. Require $|B_{\text{rec}}| = L$ and $\text{BLAKE3-256}(B_{\text{rec}}) = H_{\text{batch}}$. Parse B_{rec} into record tuples $(d_i, \text{seq}_i, t_i, C_{i,1..k}, P_i)$, first decoding $N_{\text{win}, \text{payload}}$ from the payload and requiring $N_{\text{win}, \text{payload}} = N_{\text{win}}$ from SERHDR. Require that parsing consumes exactly L bytes (no trailing bytes). Require for all i : $|C_i| = k \cdot \text{encC_len}$.
 - 4: **Recompute digests and Merkle root:** recompute $d'_i \leftarrow \text{RECDIGEST}(\text{pid}, \text{wid}, \text{seq}_i, t_i, \text{schemaID}, \text{PolicyHash}, \text{proof_suite}, C_{i,1..k})$ and require $d'_i = d_i$ for all i . Recompute R' from $\{(i-1, d'_i)\}$ and require $R' = R$ from SERHDR.
 - 5: **Proof verification:** fetch π_{win} by CID_π . Compute hdrPre and trSeed from SERHDR. Initialize transcript exactly as specified by proof_suite, and absorb trSeed as the first committed message under the suite-defined label. Compute derived commitments $\{C_{i,j}^{\text{lo}}, C_{i,j}^{\text{hi}}\}$ from $\{C_{i,j}\}$ and policy bounds θ . Verify $\text{BP.Verify}(\pi_{\text{win}}, \{C_{i,j}^{\text{lo}}, C_{i,j}^{\text{hi}}\}; 32\text{-bit}; \text{transcript}) = \text{true}$.
 - 6: on acceptance at finalized height h_{accept} on chain chainID, compute $\text{AttestHash} \leftarrow \text{AttestHash}(\text{SERHDR}, \text{chainID}, h_{\text{accept}})$.
 - 7: **off-chain (provider):** later, when publishing keys, compute $\text{KEK} \leftarrow \text{DERIVEKEK}(Z, \text{pid}, \text{wid}, \text{AttestHash})$.
-

Algorithm A.5 End-to-End Metric Lifecycle (provider $\rightarrow$ chain $\rightarrow$ IPFS $\rightarrow$ authorized reader)

Require: canonical helpers from Algorithm A.1; record processing from Algorithm A.2; proof generation from Algorithm A.3; commit/finalize from Algorithm A.4

Require: provider holds per-window K ; authorized reader holds MA-ABE keys; chain exposes a canonical proof-suite registry

- 1: **Per-record processing at the provider:** ingest $\mathbf{M}$ with t and seq ; compute commitments and record digest; encrypt plaintext containing $\mathbf{M}$ and commitment randomness; append tuple to window buffer; update Merkle frontier and bounds.
 - 2: **End-of-window payload and proof:** compute canonical payload B_{rec} and store in IPFS as a single raw block to obtain CID_{rec} ; compute Merkle root R ; serialize SERHDR ; compute Bulletproof π_{win} bound to SERHDR and PolicyHash ; store π_{win} in IPFS to obtain CID_{π} .
 - 3: **Commit:** broadcast $\text{CommitWindow}(\text{pid}, \text{wid}, \text{SERHDR})$.
 - 4: **Finalize:** broadcast $\text{FinalizeWindow}(\text{pid}, \text{wid}, \text{CID}_{\pi})$. Validators parse CID_{rec} and enforce $\text{CIDv1}/\text{raw}/\text{sha2-256}$, fetch payload and proof by CID, verify payload digest and size, require payload $N_{\text{win}, \text{payload}}$ matches header N_{win} , reject trailing bytes, recompute digests and root, and verify Bulletproof over derived commitments under transcript binding. On acceptance compute AttestHash .
 - 5: **Publish keys (post-acceptance):** publish $\text{CT}_{\text{ABE}}(Z)$ and a wrapped K bound to AttestHash so authorized readers can decrypt.
 - 6: **Authorized retrieval:** reader fetches B_{rec} by CID_{rec} , verifies digest, size, header/payload N_{win} equality, and root, decrypts Z then K , decrypts each record, recomputes commitments and record digests, and recomputes the Merkle root to match the header.
-

Algorithm A.6 AvailabilityCheck (non-decrypting; payload integrity and Merkle root)

Require: accepted header provides $(\text{CID}_{\text{rec}}, L, H_{\text{batch}}, R, N_{\text{win}})$

- 1: parse CID_{rec} and require it is CIDv1 with multicodec **raw** and multihash **sha2-256**
- 2: fetch B_{rec} by CID_{rec}

Require: $|B_{\text{rec}}| = L$ and $\text{BLAKE3-256}(B_{\text{rec}}) = H_{\text{batch}}$

- 3: parse B_{rec} into tuples $(d_i, \text{seq}_i, t_i, C_{i,1..k}, P_i)$ for $i = 1..N_{\text{win}}$ with strictly increasing seq_i , first decoding $N_{\text{win}, \text{payload}}$ from the payload and requiring $N_{\text{win}, \text{payload}} = N_{\text{win}}$ from the header

Require: parsing consumes exactly L bytes (no trailing bytes) and for all i : $|C_i| = k \cdot \text{encC_len}$

- 4: recompute $d'_i \leftarrow \text{RECDIGEST}(\text{pid}, \text{wid}, \text{seq}_i, t_i, \text{schemalD}, \text{PolicyHash}, \text{proof_suite}, C_{i,1..k})$ from header fields and require $d'_i = d_i$ for all i
- 5: set $\text{winIndex}_i \leftarrow i - 1$ and compute $R' \leftarrow \text{MERKLEAGGREGATE}(\{(\text{winIndex}_i, d'_i)\})$

Require: $R' = R$

- 6: **return OK**
-

Algorithm A.7 Off-Chain Retrieval and Decryption (Authorized Users)

- 1: **function** $\text{OFFCHAINRETRIEVALANDDECRYPTION}(\text{CID}_{\text{rec}}, \{\text{SK}_{\text{ABE}, i}\}, \text{CT}_{\text{ABE}}(Z), \text{CT}_K, \text{SERHDR}, \text{AttestHash})$

- 2: parse CID_{rec} and require it is CIDv1 with multicodec **raw** and multihash **sha2-256**

- 3: fetch B_{rec} by CID_{rec} ; parse header fields from SERHDR including $\text{pid}, \text{wid}, \text{schemalD}, \text{PolicyHash}, \text{proof_suite}, R, N_{\text{win}}, L, H_{\text{batch}}$

Require: $|B_{\text{rec}}| = L$ and $\text{BLAKE3-256}(B_{\text{rec}}) = H_{\text{batch}}$

- 4: parse records into tuples $(d_i, \text{seq}_i, t_i, C_{i,1..k}, P_i)$ for $i = 1..N_{\text{win}}$, first decoding $N_{\text{win}, \text{payload}}$ from the payload and requiring $N_{\text{win}, \text{payload}} = N_{\text{win}}$ from SERHDR

Require: parsing consumes exactly L bytes (no trailing bytes) and for all i : $|C_i| = k \cdot \text{encC_len}$

- 5: $Z \leftarrow \text{ABE.Decrypt}(\text{CT}_{\text{ABE}}(Z); \{\text{SK}_{\text{ABE}, i}\})$
- 6: $\text{KEK} \leftarrow \text{DERIVEKEK}(Z, \text{pid}, \text{wid}, \text{AttestHash})$
- 7: $K \leftarrow \text{AEAD.Dec}_{\text{KEK}}(\text{CT}_K; \text{aad} = \text{enc}_{32}(\text{PolicyHash}) \parallel \text{AttestHash}, \text{nonce} = \text{nonce}_K)$
- 8: for each i set $\text{winIndex}_i \leftarrow i - 1$ and $\text{nonce}_i \leftarrow \text{I2OSP}(\text{winIndex}_i, 12)$
- 9: for each i decrypt plaintext $(\mathbf{M}_i, r_{i,1..k}, \text{seq}'_i, t'_i) \leftarrow \text{AEAD.Dec}_K(P_i; \text{AAD}(\cdot), \text{nonce}_i)$

Require: $\text{seq}'_i = \text{seq}_i$ and $t'_i = t_i$

- 10: for each i recompute commitments $C'_{i,j}$ from $(\mathbf{M}_i, r_{i,j})$ and require $C'_{i,j} = C_{i,j}$ for all j
 - 11: for each i recompute $d'_i \leftarrow \text{RECDIGEST}(\text{pid}, \text{wid}, \text{seq}_i, t_i, \text{schemalD}, \text{PolicyHash}, \text{proof_suite}, C_{i,1..k})$ and require $d'_i = d_i$
 - 12: compute $R' \leftarrow \text{MERKLEAGGREGATE}(\{(i - 1, d'_i)\})$ and require $R' = R$
 - 13: **return OK**
 - 14: **end function**
-

Algorithm A.8 On-chain state and lifecycle (IPFS-by-CID finalize and key publication)

```
1: State: PROOF_SUITE_REGISTRY: map proof_suite  $\mapsto$  pinned verification parameters, including group parameters, ENC_C length,
   and the exact transcript procedure (construction, labels, and absorb order)
2: State: PROVIDER_STATE[pid] record with fields:
   proof_suite, policyParams, PolicyHash, bond, lastWindowID, lastSeq
3: State: PENDING_COMMIT[(pid, wid)]  $\in \{\perp \text{ or } (\text{SERHDR}, h_{\text{commit}})\}$ 
4: State: ACCEPTED[(pid, wid)]  $\in \{\perp \text{ or } (\text{SERHDR}, \text{AttestHash}, h_{\text{accept}})\}$ 
5: State: POLICY_REGISTRY: map PolicyHash  $\mapsto \theta$ , append-only
6: Constant:  $T_{\text{finalize}}$  = maximum allowed chain-height delta between commit and finalize (deployment-fixed)
7: function REGISTER_PROVIDER(pid, proof_suite, policyParams, bond)
Require: PROVIDER_STATE[pid] is empty and proof_suite  $\in$  PROOF_SUITE_REGISTRY
8:    $\theta \leftarrow$  canonical serialization derived from policyParams
9:   PolicyHash  $\leftarrow$  POLICY_HASH( $\theta$ )
10:  PROVIDER_STATE[pid]  $\leftarrow$  (proof_suite, policyParams, PolicyHash, bond, 0, 0)
11:  POLICY_REGISTRY[PolicyHash]  $\leftarrow \theta$ 
12: end function
13: function UPDATE_POLICY(pid, policyParams)
Require: authorized caller for pid
14:    $\theta \leftarrow$  canonical serialization from new policyParams
15:   PROVIDER_STATE[pid].policyParams  $\leftarrow$  policyParams
16:   PROVIDER_STATE[pid].PolicyHash  $\leftarrow$  POLICY_HASH( $\theta$ )
17:   POLICY_REGISTRY[PolicyHash]  $\leftarrow \theta$ 
18: end function
19: function COMMIT_WINDOW(pid, wid, SERHDR)
20:   parse SERHDR exactly as in Algorithm A.1 into header fields including  $R, N_{\text{win}}, \text{PolicyHash}, \text{proof\_suite}, L, H_{\text{batch}}, \text{CID}_{\text{rec}}$ 
Require: proof_suite = PROVIDER_STATE[pid].proof_suite and PolicyHash = PROVIDER_STATE[pid].PolicyHash
Require: POLICY_REGISTRY[PolicyHash] exists
Require:  $N_{\text{win}} > 0$ , wid = PROVIDER_STATE[pid].lastWindowID + 1, and seq_min > PROVIDER_STATE[pid].lastSeq
Require:  $L \leq L_{\text{max}}$ 
Require: PENDING_COMMIT[(pid, wid)] =  $\perp$ 
21:    $h_{\text{commit}} \leftarrow$  current height
22:   PENDING_COMMIT[(pid, wid)]  $\leftarrow$  (SERHDR,  $h_{\text{commit}}$ )
23: end function
24: function FINALIZE_WINDOW(pid, wid, CID $_{\pi}$ )
Require: PENDING_COMMIT[(pid, wid)] exists as (SERHDR,  $h_{\text{commit}}$ )
25:    $h_{\text{now}} \leftarrow$  current height
Require:  $h_{\text{now}} - h_{\text{commit}} \leq T_{\text{finalize}}$ 
26:   parse SERHDR and compute hdrPre and trSeed as in Algorithm A.1
27:   parse CID $_{\text{rec}}$  from SERHDR and require it is CIDv1 with multicodec raw and multihash sha2-256
28:   fetch  $B_{\text{rec}}$  by CID $_{\text{rec}}$  from SERHDR and require  $|B_{\text{rec}}| = L$  and BLAKE3-256( $B_{\text{rec}}$ ) =  $H_{\text{batch}}$ 
29:   parse tuples into  $(d_i, \text{seq}_i, t_i, C_{i,1..k}, P_i)$ , first decoding  $N_{\text{win,payload}}$  from the payload and requiring  $N_{\text{win,payload}} = N_{\text{win}}$ 
   from SERHDR
Require: parsing consumes exactly  $L$  bytes (no trailing bytes) and for all  $i$ :  $|C_i| = k \cdot \text{encC\_len}$ 
30:   recompute  $d'_i \leftarrow \text{REC\_DIGEST}(\text{pid}, \text{wid}, \text{seq}_i, t_i, \text{schemalD}, \text{PolicyHash}, \text{proof\_suite}, C_{i,1..k})$  and require  $d'_i = d_i$  for all  $i$ 
31:   recompute  $R'$  with canonical rules from  $\{(i-1, d'_i)\}$ , and require  $R' = R$ 
32:   fetch  $\pi_{\text{win}}$  by CID $_{\pi}$ . Initialize transcript exactly as specified by proof_suite, and absorb trSeed as the first committed
   message under the suite-defined label.
33:   compute derived commitments  $\{C_{i,j}^{\text{lo}}, C_{i,j}^{\text{hi}}\}$  from  $\{C_{i,j}\}$  and policy bounds  $\theta$  from POLICY_REGISTRY[PolicyHash]
34:   verify Bulletproof for the derived commitments under 32-bit ranges and the pinned proof suite
Require: verification is true
35:    $h_{\text{accept}} \leftarrow$  current height; chainID  $\leftarrow$  network id
36:   AttestHash  $\leftarrow$  AttestHash(SERHDR, chainID,  $h_{\text{accept}}$ )
37:   ACCEPTED[(pid, wid)]  $\leftarrow$  (SERHDR, AttestHash,  $h_{\text{accept}}$ )
38:   PROVIDER_STATE[pid].lastSeq  $\leftarrow$  seq_max and PROVIDER_STATE[pid].lastWindowID  $\leftarrow$  wid
39:   PENDING_COMMIT[(pid, wid)]  $\leftarrow \perp$ 
40: end function
41: function PUBLISH_KEYS(pid, wid, CT $_{\text{ABE}}(Z)$ , CT $_K$ )
Require: ACCEPTED[(pid, wid)] exists as (SERHDR, AttestHash,  $h_{\text{accept}}$ )
42:   store (CT $_{\text{ABE}}(Z)$ , CT $_K$ ) on-chain for (pid, wid)
43: end function
```

Algorithm A.9 Challenge (optional, deployment-specific; evidence of misbehavior)

- 1: **Context:** This procedure is intended to run off-chain; only a verdict or a succinct proof derived from it need be posted on-chain.
 - 2: **Note:** Since acceptance verifies a window-level range proof, range-policy violations should not be possible for accepted windows under the pinned proof suite. Deployments may instead use challenges for omission evidence, operational disputes, or alternative accountability mechanisms.
 - Require:** accepted (pid, wid) with SERHDR, R , PolicyHash, and CID_{rec}
 - Require:** challenger provides record index winIndex, decrypted plaintext $(\mathbf{M}, r_{1..k}, \text{seq}, t)$, and the commitments $(C_1, \dots, C_k)$ as they appear in the payload
 - 3: recompute commitments from $(\mathbf{M}, r_{1..k})$ and require equality to $(C_1, \dots, C_k)$
 - 4: recompute record digest d using RECDIGEST and require that d appears at winIndex in the committed payload B_{rec} fetched by CID_{rec}
 - 5: apply deployment-defined dispute handling (for example, auditing, omission checks, or provider penalties) or reject challenge
-

Interoperability and minimal test vectors

Endianness and widths. All fixed-width integer encodings are big-endian via I2OSP. The provider id pid, the policy digest PolicyHash, the payload digest H_{batch} , the Merkle root R , and header hashes are always 32 bytes. Sequence numbers and timestamps are encoded as 8-byte unsigned integers. The proof suite identifier, schema identifier, and chain identifier are encoded as 4-byte unsigned integers.

Header determinism. The header bytes SERHDR are uniquely determined by the window identifiers, bounds, Merkle root, policy digest, proof suite id, and the committed payload tuple $(L, H_{\text{batch}}, \text{CID}_{\text{rec}})$. Validators must reject any finalize attempt that yields payload bytes that do not match the committed (L, H_{batch}) pair, even if the CID fetch succeeds, because this pair is the application-level integrity check that makes validator behavior deterministic across IPFS implementations and block encodings. Validators MUST also enforce CID version/codec/hash constraints, require that payload $N_{\text{win}, \text{payload}}$ equals header N_{win} , and reject trailing bytes (parsing must consume exactly L bytes).

Transcript determinism. The transcript seed trSeed is deterministically derived from SERHDR via hdrPre and PolicyHash. Implementations must initialize the Bulletproof transcript with trSeed as the first committed message, and must use the pinned proof suite’s transcript construction, labels, absorb order, and encoding rules for commitments and bounds so that independent implementations derive the same Fiat–Shamir challenges.

Copyright and Licensing Statement

© Johan B, 2025. All rights reserved. This work is licensed under the following terms: Redistribution, reproduction, and use of this work for academic and educational purposes are permitted, provided that proper credit is given to the original author(s) as specified below. Commercial use of this work, in whole or in part, is strictly prohibited without the explicit written consent of the author(s). The author(s) retain exclusive rights to commercialize this work, including licensing, distribution, and monetization of its content or derivatives. For permissions or commercial inquiries, please contact the author(s) at ch9chatgpt@gmail.com. This document and its associated rights are protected under applicable copyright laws. Unauthorized commercial use, distribution, or reproduction may result in legal action.